Core Monitor

Feasibility study: using and monitoring all processor cores – project report with findings, measurements and installation notes

≈ 100×
faster than a single Python process (Rust SIMD, 48 threads)

11
CPU computation methods producing a bit-identical image (Python, C#, Rust, C, C++, assembly)

2.1×
maximum speedup for pi – Amdahl's law in practice

As of: 8 November 2026 · Reference machine: AMD Ryzen Threadripper 3960X (24 cores / 48 threads), 64 GB RAM, NVIDIA GeForce RTX 3090, Windows 11

Contents

1. Goal and result at a glance
2. Getting started, structure and features
3. Findings: Mandelbrot, scaling, placement and SMT
4. Language comparison: Python, C#, Rust, C, C++, assembly – and SIMD
5. A smarter algorithm (cardioid) and the graphics card (GPU)
6. Pi and Amdahl's law
7. Bugs found and fixed
8. Distribution: Full and Lite packages, installation
9. Limitations and ideas
10. Glossary

1. Goal and result at a glance

Goal: a program for using and monitoring all processor cores, operated through a local web page, with a graphical display of the load on every single core. It is a feasibility study to explore what is possible.

Result: Core Monitor shows each of the 48 threads live (grouped by core and cache group), generates load on demand and computes two real parallel tasks – the Mandelbrot set and the number π – in many variants. This yields reliable measurements of what parallelisation delivers and where its limits lie.

> **Key findings**
> - **Tasks that split well scale strongly:** Mandelbrot runs 18–28× faster with 48 threads than with one.
> - **The programming language hardly matters:** numba (Python), C#, Rust, C, C++ and hand-written assembly are equally fast per core (± a few percent). The differences lie in everything around it: because of the GIL, Python has to use processes instead of threads (slow start-up, data transfer).
> - **SIMD gives the biggest gain per core:** 4 pixels at once (AVX2) = 3.7× faster.
> - **A better algorithm beats more cores:** the cardioid test makes the full view 6–9× faster on 1 thread.
> - **Serial parts limit everything (Amdahl):** for π the speedup tops out at ≈ 2×, even with 48 cores.
> - **The GPU** (RTX 3090) is up to 5× faster than the best CPU variant in *float*, but slower in *double*.

Reference machine

Component	Specification
Processor	AMD Ryzen Threadripper 3960X: 24 cores / 48 threads (SMT), 8 cache groups (CCX) of 3 cores each with 16 MB L3, AVX2 + FMA, no AVX-512, max. 3.8 GHz
Memory	64 GB
Graphics card	NVIDIA GeForce RTX 3090, 24 GB, Direct3D 11 compute, double support (1/64 of float performance)
Software	Windows 11, Python 3.11.9, numba 0.68, numpy 2.4, gmpy2 2.3, .NET SDK 10, Rust 1.99, Visual Studio 2026 (MSVC 14.51)

2. Getting started, structure and features

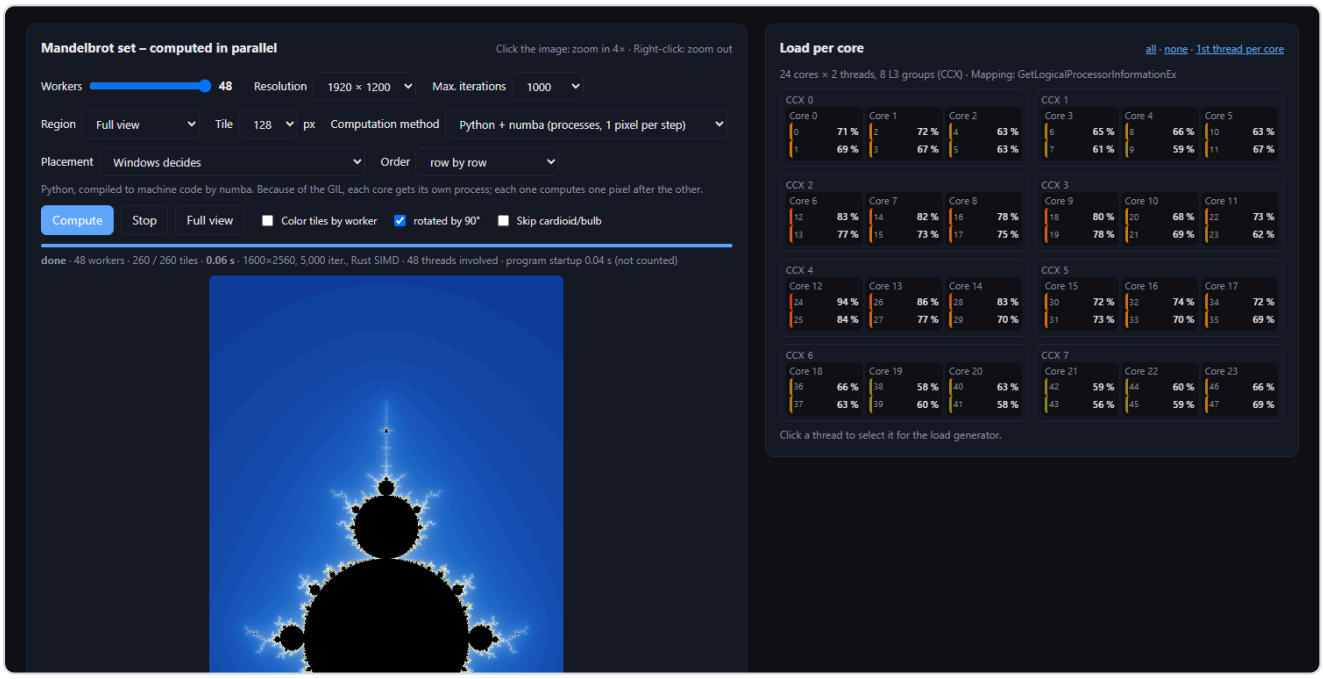
Starting: double-click `start.bat`. The browser opens `http://127.0.0.1:8000` (only if no page is open yet; an open page reloads itself after a restart). The server is deliberately **reachable only from this PC** (127.0.0.1), not from the network.

Language: the **DE | EN** switch changes the page between German and English. The choice is remembered; without a saved choice the page starts in English. Server messages follow the selected language. The environment variable `KM_LANG` (`de` or `en`) sets the start language.

Features of the user interface

- **System box:** hardware detected at start-up (CPU, cores, cache groups, instruction sets, RAM, graphics cards) and unavailable computation methods with the reason.

- **Load per core:** all threads live (every second), grouped by cache group → core → thread, with a 60-second history; plus total CPU, clock speed, RAM.
- **Heatmap:** all threads over 2 minutes – shows how load is distributed over time and across the chip.
- **Mandelbrot:** 13 computation methods (see section 4), 1–48 workers, resolution, iterations, view region, tile size, placement (CPU affinity), order ("demanding tiles first"), cardioid test, live tiles, colouring by worker, click to zoom, 90° rotation.
- **Scaling test:** the same image with 1 … 48 workers (median of 1/3/5 runs), chart as speedup or run time, comparison of all variants; results as CSV (Excel) and JSON.
- **Pi:** 1,000 to 100 million decimal places, phase display, verification against mpmath, download; scaling test with breakdown by phase.
- **Load generator** (any number of threads, intensity, duration, affinity) and **prime number benchmark**.
- **Activity indicator:** spinner on every running computation and a real heartbeat ("no response for … s" if the server does not answer).
- **Language switch DE | EN** for the interface and server messages (choice is remembered).
- All explanations can be expanded (collapsed on load).



Part of the user interface: Mandelbrot section and load per core (8 cache groups × 3 cores × 2 threads).

Structure (files)

File / folder	Contents
`server.py`	small launcher (sets up the environment, starts `app.py`) – kept tiny on purpose because worker processes reload it
`app.py`	web server (FastAPI), measurement (psutil), WebSocket, API, load generator, prime number benchmark
`hardware.py`	hardware detection (CPU, topology, instruction sets, graphics cards), scaling steps, simulation for tests
`mandelbrot.py`	Mandelbrot: numba/numpy computation methods, process pool, integration of the external programs, scaling test
`pi.py`	Pi: Chudnovsky with binary splitting (gmpy2), phase timing, verification, scaling test
`worker_util.py`, `lastgen.py`	lightweight modules for worker processes
`static/`	user interface: `index.html`, `app.js` (core view, heatmap), `mandel.js`, `pi.js`, `style.css`
`csharp/`, `rust/`, `native/`	comparison programs: C#, Rust, C, C++, assembly (`asm/kernel.asm`), GPU (`gpu/mandel.hlsl`)
`benchmarks/`	measurement results (CSV/JSON), pi result and mpmath reference
`export_*.bat`, `tools/`	building the distribution packages, installation guides

3. Findings: Mandelbrot, scaling, placement and SMT

The Mandelbrot set is an "embarrassingly parallel" task: the image is split into tiles (128 × 128 pixels) that are computed independently of each other. A process pool hands them out dynamically: whoever is finished fetches the next tile. Standard measurement: 1600 × 2560 pixels (rotated 90°), 5,000 iterations, full view, median of 3 runs.

Python + numba with 1 … 48 processes

Workers	1	2	4	8	12	16	24	32	48
Time (s)	6.18	3.27	1.72	0.93	0.72	0.56	0.51	0.40	0.34
Speedup	1.0×	1.9×	3.6×	6.6×	8.5×	11.0×	12.1×	15.5×	18.4×

- **Python uses multiple cores only via processes** (because of the GIL), not via threads. Starting 48 processes takes 2–6 s on Windows; the pool is therefore kept alive, and the start-up time is reported separately.
- **Almost linear up to 8 workers**, after which the curve flattens. The main reason is **not** SMT but the short run time: with 48 workers one image takes only 0.3 s – at that point the overhead and the last tiles that everyone waits for dominate. With 4 times the work (20,000 iterations) the result with 48 workers rises to **24.8×**; the CPU time of all workers stays the same (≈ 24 s), what is lost is pure waiting time.
- **Prime numbers** up to 3 million (first prototype): 7.0 s serial, 0.7 s with 48 processes (10×).

Placement (CPU affinity) and SMT

Workers	2	4	8	12	16	24	48
Windows decides (s)	3.27	1.72	0.93	0.72	0.56	0.51	0.34
1 thread per core first (s)	3.14	1.64	0.90	0.66	0.55	0.49	0.34
compact, both threads of a core (s)	3.39	1.76	0.96	0.71	0.58	0.49	0.35

- **Placement makes little difference:** "1 thread per core first" is 3–10 % faster with 2–16 workers; with 48 workers all are equal.
- **Windows packs a few workers tightly together:** in tests, 4 workers ended up on threads 24–27 – i.e. on 2 cores using both threads each, although 22 cores were idle.
- **SMT barely slows Mandelbrot down:** 2 workers on one core are only 8 % slower than on two cores (11 % with SIMD). The reason: every computation step waits for the previous one; a single thread leaves most of the execution units idle, and the second one fills the gaps.

"Demanding tiles first"

Tiles in the black interior are demanding to compute (they run up to the maximum iteration count). A rough cost estimate (8 × 8 samples per tile, ≈ 15 ms, included in the time) moves them to the front so that no long tile is left over at the end.

Workers	16	24	32	48
numba row by row (s)	0.56	0.51	0.40	0.34
numba demanding tiles first (s)	0.54	0.40	0.36	0.29

Result: 10–22 % faster with 24–48 workers (18.4× → 21.8×), no effect with few workers. With very short run times (SIMD, ≈ 70 ms) the estimate takes longer than it saves.

4. Language comparison – and SIMD

The 13 computation methods

"Computation method" here means one of 13 variants for computing the same Mandelbrot image – not a processor core. Each method can use all cores of the PC if desired; the GPU methods run on the graphics card.

Computation method	Type	Description
Python + numba	processes, 1 pixel per step	Python, compiled to machine code by numba; one process per core because of the GIL
Python + numpy	processes, array operations	pure Python with numpy, whole tiles as numeric arrays; slowest variant
C# / .NET	threads, 1 pixel per step	C# (.NET 10) with real threads in one process
C# / .NET + SIMD	threads, 4 pixels at once	like C#, 4 pixels at once per thread (AVX2)
Rust	threads, 1 pixel per step	Rust with real threads and a shared tile queue
Rust + SIMD	threads, 4 pixels at once	Rust with AVX2 instructions
C	threads, 1 pixel per step	C with Windows threads (CreateThread)
C + SIMD	threads, 4 pixels at once	C with AVX2 intrinsics
C++	threads, 1 pixel per step	C++ with std::thread and std::atomic
C++ + SIMD	threads, 4 pixels at once	C++ with AVX2 intrinsics
Assembly	threads, 4 pixels at once	hand-written x64 assembly (AVX2) for the inner loop; threads in C
GPU (double)	graphics card, double precision	Direct3D compute shader, one GPU thread per pixel; double precision, individual pixels differ (driver)
GPU (float)	graphics card, single precision	like GPU (double), much faster, image differs slightly

The same task in six languages or tools, each with real threads (except Python: processes) and with SIMD (AVX2: 4 pixels at once). All variants produce a **bit-identical image** (0 pixels difference, checked in several views): the same formula, the same order of computation steps, no FMA (fused multiply-add would change the last digit).

Threads	numba	C#	Rust	C	C++	C# SIMD	Rust SIMD	C SIMD	C++ SIMD	Assembly
1	6.18	6.09	6.21	6.08	6.06	1.65	1.63	1.66	1.67	1.69
8	0.93	0.83	0.85	0.81	0.80	0.23	0.22	0.22	0.23	0.23
24	0.51	0.40	0.40	0.39	0.39	0.11	0.10	0.11	0.11	0.11
48	0.34	0.23	0.22	0.23	0.24	0.070	**0.062**	0.070	0.071	0.072
Start	2–6 s	0.14	0.04	0.10	0.10	0.14	0.04	0.10	0.10	0.10

Times in seconds, 1600 × 2560, 5,000 iterations, row by row, median of 3. "Start" = program or process start-up (not included).



Run time (logarithmic) of all languages: two bundles – without SIMD (top) and with SIMD (bottom, dashed); only numba (blue) stands apart.

- **The language hardly matters:** all compilers (MSVC, LLVM for Rust and numba, RyuJIT for C#) generate practically the same machine instructions for the inner loop. The limit is set by the processor: each step waits for the result of the previous one.
- **Hand-written assembly** (`native/asm/kernel.asm`, extensively commented) is no faster than the compiler.
- **Real threads instead of processes** give 23–28× instead of 18× at 48 threads, and start-up in 0.04–0.14 s instead of seconds.
- **SIMD** makes a single thread **3.7× faster** – the largest single gain. Best result: **Rust SIMD with 48 threads, 0.062 s** ≈ 100× faster than a single Python process. This processor does not have AVX-512.
- Going further would only be possible with tricks that work equally in every language: interleaving several pixel groups, FMA (changes the image in the last digit) or float instead of double (inaccurate when zooming).

5. A smarter algorithm and the graphics card

Skipping the cardioid and the period-2 bulb

Points inside the main cardioid and inside the bulb to its left are certainly in the set – a simple formula detects them ($q = (x - ¼)^2 + y^2$; in the set if $q \cdot (q + x - ¼) \le y^2/4$ or $(x + 1)^2 + y^2 \le 1/16$). Without the test, exactly these most demanding points run up to the maximum iteration count. The image stays **exactly the same** (checked in all computation methods, including right at the edges).

View	Rust, 1 thread	Rust, 48 threads	Rust SIMD, 1 thread	Rust SIMD, 48 threads
Full view	6.38 → 0.73 s (8.7×)	0.24 → 0.11 s (2.3×)	1.67 → 0.29 s (5.7×)	0.068 → 0.032 s (2.1×)
Edge of the cardioid	27.7 → 2.0 s (13.6×)	0.80 → 0.21 s (3.7×)	6.9 → 0.64 s (10.8×)	0.21 → 0.053 s (3.9×)
Seahorse valley	unchanged	unchanged	unchanged	unchanged

> **A better algorithm beats more cores:** 1 thread with the test is faster than 8 threads without it. With many threads the gain is smaller because the remaining work is unevenly distributed and the run time is down to hundredths of a second.

Graphics card (RTX 3090)

A Direct3D 11 compute shader (`native/gpu/mandel.hlsl`) computes each pixel in its own GPU thread, in strips of 128 rows (Windows aborts GPU jobs after ≈ 2 s). The CPU then computes the colours. Pure computation time without image transfer:

Setting	Cardioid	Rust SIMD (48 thr.)	GPU double	GPU float
1600 × 2560, 5,000 it.	no	0.067 s	0.173 s (0.4×)	0.028 s (2.4×)
1600 × 2560, 5,000 it.	yes	0.031 s	0.101 s (0.3×)	0.027 s (1.2×)
1600 × 2560, 20,000 it.	no	0.238 s	0.575 s (0.4×)	0.046 s (5.2×)
3840 × 6144, 5,000 it.	no	0.288 s	0.846 s (0.3×)	0.109 s (2.6×)
3840 × 6144, 5,000 it.	yes	0.078 s	0.388 s (0.2×)	0.087 s (0.9×)

- **float:** up to 5× faster than the best CPU variant; the GPU itself needs only 0.01–0.03 s, the rest is copying back and colouring. But less accurate: 0.3 % of pixels differ in the full view, 4–17 % in deep zooms.
- **double:** slower than the CPU (0.2–0.4×) – the RTX 3090 computes double at only 1/64 of float speed (compute cards such as the A100/H100 are much better). Here too, 0.01–0.5 % of pixels differ; FMA is not the cause (ruled out by a cross-check), the graphics driver compiles the shader in its own way.
- **With the cardioid test** the CPU catches up and is even faster for the 4K image. GPU start-up ≈ 0.45 s. GPU load does not appear in the CPU heatmap.

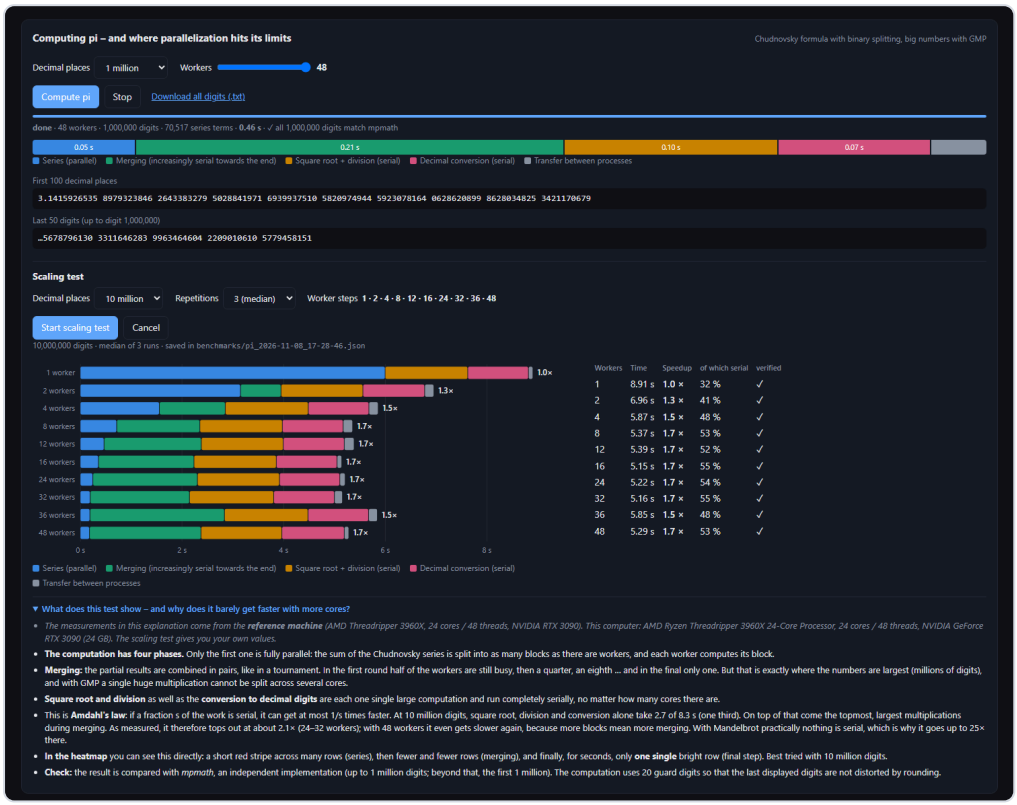
6. Pi and Amdahl's law

π is computed with the Chudnovsky formula and binary splitting (GMP via gmpy2). Only the summation of the series is fully parallel; it is followed by merging (less and less parallel towards the top), square root and division, and the conversion to decimal digits – each a single large computation.

Workers	1	2	4	8	16	24	32	48
Total (s)	8.33	6.41	5.31	4.84	4.57	3.95	3.97	4.72
Speedup	1.0×	1.3×	1.6×	1.7×	1.8×	2.1×	2.1×	1.8×
Series, parallel (s)	5.56	3.10	1.47	0.73	0.36	0.18	0.16	0.17
Merging (s)	–	0.55	1.12	1.42	1.52	1.38	1.38	1.80
Square root + division (s)	1.57	1.59	1.56	1.52	1.56	1.36	1.40	1.58
Decimal conversion (s)	1.20	1.16	1.16	1.16	1.14	1.03	1.03	1.16

10 million decimal places, median of 3. All runs match mpmath (fully verified up to 1 million digits).

> **Amdahl's law:** if a fraction s of the work is serial, the speedup is at most 1/s. The parallel part shrinks from 5.6 s to 0.17 s (≈ 33×), but ≈ 2.7 s for square root, division and conversion remain on a single core – plus the largest multiplications during merging. Result: at most ≈ 2.1×; with 48 workers it even gets slower again (more merging, more data transfer). In the heatmap you first see a red band across all threads, then fewer and fewer, and for several seconds just a single bright row.



Pi section: phase bars, first and last digits, scaling test with stacked bars per phase.

Measured optimisations: one block per worker instead of four (4.6 instead of 5.4 s), merging distributed across the workers instead of in the main process (4.6 instead of 5.8 s), and one of the largest multiplications is dropped in the last step. A C# or Rust counterpart was deliberately not built: without GMP the big-number libraries would be many times slower – the comparison would measure the library, not the parallelisation.

7. Bugs found and fixed

Problem	Cause	Fix
Live view froze after "Generate load – Start"	Status incomplete during process start-up → an error silently ended the measurement loop	set status in advance, measurement loop catches errors, start/stop/status under a lock
"Paging file too small", workers crashed	numpy/OpenBLAS allocated buffers for 48 threads in every worker (≈ 800 MB per worker, 37.6 GB with 48); every worker loaded the whole web server	`OPENBLAS_NUM_THREADS=1`, tiny launcher, lightweight worker modules: 62 MB per worker (2.9 GB with 48), web server 88 instead of 834 MB
Server unresponsive for seconds during large pi computations	GMP does not release the Python GIL	serial pi steps in a worker process; measured max. 0.22 s response time at 50 million digits
Orphaned worker processes after the server exits	Windows does not terminate pool children along with the parent	workers watch their parent process and exit on their own
First run used only 21 of 48 workers	pool was not yet complete at start-up	wait until all workers have responded
C#/Rust slowed down at short run times	tiles transferred as base64 text	binary transfer (0.116 → 0.070 s with SIMD)
Wrong speedup in the scaling test	plan without the "1 worker" step	1 worker is always included
Right after unpacking, individual computation methods and GPU detection failed with "Access denied"	antivirus software locks freshly unpacked .exe files for a few seconds on first start	start is retried up to 10 times at one-second intervals; if an .exe is missing from the Full package, the page names antivirus software as the likely cause; note in chapter 8 and INSTALLATION.txt
Old server bound to 0.0.0.0 (reachable from the network)	first prototype	address fixed to 127.0.0.1
Outdated page files in the browser	browser cache	cache headers, automatic reload on server restart or file change
Package started foreign builds or failed without tools	timestamps after copying	use the bundled programs if tools are missing; tolerance of 1 minute

8. Distribution: Full and Lite packages

	Full package	Lite package
Build with	`export_full.bat`	`export_lite.bat`
Result	`export\CoreMonitor_Full_<date>.zip`	`export\CoreMonitor_Lite_<date>.zip`
Size	ZIP 104 MB, unpacked 281 MB	ZIP 0.1 MB, unpacked 0.3 MB
Contents	Python 3.11.9 (embedded) with all packages; all 13 computation methods ready-built (C/C++/assembly/GPU with static runtime library, precompiled shaders, Rust statically linked, C# as a self-contained .exe); source code; mpmath reference	source code and instructions only
Target PC needs	only Windows 10/11 (64-bit) – no Python, no internet, no admin rights	Python 3.11+ and internet once; optionally .NET SDK, Rust, Visual Studio C++
Tested	different folder, empty PATH: all 13 computation methods, pi, load, scaling test; simulated small PC (8 threads, no AVX2, no GPU)	with all tools (everything compiled, 13 computation methods, start-up 27 s) and with Python only (2 computation methods, 11 greyed out with a note)

Your own measurement data, settings, logs and intermediate files are not included in either package. Both contain `README.md` (English) and `README.de.md` (German), `INSTALLATION.txt` (English) and `INSTALLATION.de.txt` (German), and both project reports as PDF: `Project_Report_Core_Monitor.pdf` (English) and `Projektbericht_Parallelisierung.pdf` (German).

> **Note on antivirus software: false positives are possible.** Antivirus software (e.g. Microsoft Defender, Trend Micro) and Windows SmartScreen may raise an alarm about this program or block parts of it – a false positive. The triggers are behaviour patterns that malware also shows: self-compiled `.exe` files without a signature or reputation, freshly unpacked files, up to 48 processes started at the same time and 100 % load on all cores (similar to a crypto miner). **Signs:** "Access denied" or a slow first start; `.exe` in quarantine (computation method greyed out with a note); SmartScreen warning. **What to do:** wait briefly or restart `start.bat` (the monitor retries blocked starts for up to 10 s); before unpacking, tick "Unblock" in the ZIP's Properties; restore the file from quarantine; add the folder as an exclusion – only if it comes from a trustworthy source (built yourself or obtained directly from the author); to double-check, scan the `.exe` files on VirusTotal or compile the Lite package yourself. Do not switch off your antivirus software. numba, numpy and pi also work without the `.exe` files.

Installation of the Lite package (step by step)

Step	Needed for	How	Check
1. Python 3.11+ (required)	everything	python.org → "Windows installer (64-bit)", tick "Add python.exe to PATH"; or `winget install Python.Python.3.11`	`python --version`
2. Python packages	everything	done automatically by `start.bat` (fastapi, uvicorn, psutil, numpy, gmpy2, mpmath, numba)	page starts
3. .NET SDK 10 (optional)	C#, C# SIMD	dotnet.microsoft.com or `winget install Microsoft.DotNet.SDK.10`	`dotnet --version`
4. Rust (optional)	Rust, Rust SIMD	rustup.rs or `winget install Rustlang.Rustup` (needs the linker from step 5)	`cargo --version`
5. Visual Studio C++ (optional)	C, C++, assembly, GPU	Build Tools or Community, workload "Desktop development with C++" (cl, ml64, Windows SDK with fxc)	`native\build.bat`
6. Graphics driver (optional)	GPU	current driver from NVIDIA, AMD or Intel (Direct3D 11)	"System" box

The page detects the CPU and graphics card at start-up; SIMD and assembly require AVX2. Anything missing appears greyed out with the reason. The Full package needs none of this: unpack, double-click `start.bat`.

9. Limitations and ideas

- **Windows only** (core mapping, affinity, Direct3D); Linux/macOS only partially supported and not tested.
- **Above 64 threads** (multiple processor groups) only the "Windows decides" placement is offered.
- **GPU images** differ slightly (double 0.01–0.5 %, float up to 17 % in zooms); double is slow on gaming graphics cards.
- **Further ideas:** interleaving several pixel groups (hiding latency), smaller tiles only at the end, periodicity checking, CUDA/OpenCL instead of Direct3D, multi-precision Mandelbrot for deep zooms, running on a machine with AVX-512 or a compute GPU for comparison.

Measurements come from the reference machine; on other machines the built-in scaling test provides their own figures.

10. Glossary

Alphabetical, with a link to further reading.

0.0.0.0	Server setting "reachable on all network adapters". The monitor deliberately does not use it, only 127.0.0.1.
127.0.0.1 (localhost)	Address of your own PC. The server listens only there and is therefore not reachable from other computers. https://en.wikipedia.org/wiki/Localhost
Amdahl's law	The part of a program that cannot run in parallel limits the possible speedup, no matter how many cores are used (see Pi). https://en.wikipedia.org/wiki/Amdahl%27s_law
API	Interface through which the page (or a script) controls the server, e.g. /api/mandel/start. https://en.wikipedia.org/wiki/API
Assembler (MASM)	Processor instructions in readable form, written by hand. MASM is the Microsoft Macro Assembler. https://learn.microsoft.com/en-us/cpp/assembler/masm/microsoft-macro-assembler-reference
AVX-512	Successor of AVX2 with 512-bit registers (8 doubles at once). The Threadripper 3960X does not have it; some newer processors do. https://en.wikipedia.org/wiki/AVX-512
AVX2	SIMD instruction set of current x86 processors; 256-bit registers hold 4 floating-point numbers (double), i.e. 4 pixels per instruction. https://en.wikipedia.org/wiki/Advanced_Vector_Extensions
base64	Method to send binary data (here image tiles) as plain text; about one third larger than the original. https://en.wikipedia.org/wiki/Base64
Binary splitting	Technique that combines the series terms pairwise into large fractions; the parts can be computed in parallel, the final division cannot. https://en.wikipedia.org/wiki/Binary_splitting
Bit-identical	Every byte of the result is identical – here: 0 pixels difference between the computation methods.
C / C++	Classic languages close to the hardware. Compiled here with the Microsoft compiler (MSVC) from Visual Studio. https://learn.microsoft.com/en-us/cpp/
C# / .NET	Programming language and runtime from Microsoft; real threads, compiled to machine code at run time. https://learn.microsoft.com/en-us/dotnet/csharp/
Cache (L1/L2/L3)	Fast buffer memory inside the processor. L1/L2 belong to one core, L3 is shared by the cores of a CCX. https://en.wikipedia.org/wiki/CPU_cache

Cardioid and period-2 bulb	The heart-shaped main area and the large disc to its left certainly belong to the Mandelbrot set. A short formula detects these points without iterating them up to the maximum. https://en.wikipedia.org/wiki/Mandelbrot_set#Main_cardioid_and_period_bulbs
CCX (Core Complex)	On AMD Zen processors a group of cores sharing one L3 cache; the 3960X has 8 CCXs with 3 cores each. Called "L3 group" in the monitor. https://en.wikipedia.org/wiki/Zen_2
Chudnovsky formula	Very fast converging series for Pi: about 14 new digits per term. https://en.wikipedia.org/wiki/Chudnovsky_algorithm
Compiler switches /O2, /arch:AVX2, /fp:precise, /MT	Optimise, allow AVX2 instructions, compute exactly as written (no FMA), link the runtime library statically (no extra DLLs needed). https://learn.microsoft.com/en-us/cpp/build/reference/compiler-options-listed-by-category
Core (processor core)	Independent execution unit inside the processor. The Threadripper 3960X has 24 cores. https://en.wikipedia.org/wiki/Multi-core_processor
CPU affinity (placement)	Defines on which threads a process may run. The monitor uses it to compare "1 thread per core first" with "compact". https://en.wikipedia.org/wiki/Processor_affinity
"Demanding tiles first"	Order in which a rough up-front estimate puts the most compute-intensive tiles first, so that no long tile is left running alone at the end.
Digital signature (code signing)	Electronic signature of a publisher on an .exe; signed, widely used programs trigger warnings less often. The programs of Core Monitor are not signed. https://en.wikipedia.org/wiki/Code_signing
Direct3D 11 compute shader / HLSL	Program that runs on the graphics card; written in HLSL, launched via Direct3D 11 (works with any current graphics card on Windows). https://learn.microsoft.com/en-us/windows/win32/direct3d11/direct3d-11-advanced-stages-compute-shader
double / float	Floating-point numbers with double (≈ 16 digits) or single precision (≈ 7 digits) according to IEEE 754. Gaming graphics cards are much faster with float. https://en.wikipedia.org/wiki/IEEE_754
DXGI	Windows interface the monitor uses to detect the installed graphics cards and their memory. https://learn.microsoft.com/en-us/windows/win32/direct3ddxgi/dx-graphics-dxgi
Embedded Python	Official, lean Python distribution that can be shipped in a folder without installation. Included in the "Full" package. https://docs.python.org/3/using/windows.html#the-embeddable-package
False positive	Antivirus software considers a harmless program dangerous because it behaves similarly to malware (e.g. unsigned .exe, full load on all cores). See "Note on antivirus software". https://en.wikipedia.org/wiki/False_positives_and_false_negatives
FastAPI / uvicorn	Python building blocks of the web server: FastAPI describes the interfaces, uvicorn serves them. https://fastapi.tiangolo.com/
FMA (fused multiply-add)	Multiplication and addition in one step with a single rounding. Faster, but changes the last digit – and thus individual pixels. https://en.wikipedia.org/wiki/Multiply%E2%80%93accumulate_operation

GIL (Global Interpreter Lock)	Lock in standard Python: only one thread executes Python code at a time. That is why the monitor uses processes instead of threads for Python. https://docs.python.org/3/glossary.html#term-global-interpreter-lock
gmpy2 / GMP	Library for very large numbers with any number of digits; the basis of the Pi computation. https://gmpy2.readthedocs.io/
GPU (graphics card)	Processor with thousands of simple execution units; ideal when a huge number of identical calculations is needed (e.g. one per pixel). https://en.wikipedia.org/wiki/Graphics_processing_unit
Heat map	Coloured tile view: one field per thread, colour = current load. https://en.wikipedia.org/wiki/Heat_map
Intrinsics	Functions in C/C++/Rust that map directly to one processor instruction (e.g. AVX2); SIMD without assembler. https://learn.microsoft.com/en-us/cpp/intrinsics/compiler-intrinsics
Iterations (max_iter)	The maximum number of times $z \to z^2 + c$ is repeated. More iterations = finer edges, but more computing time (especially in the black interior).
JIT (just-in-time compilation)	Code is compiled to machine code only while the program runs; the first call therefore takes longer. https://en.wikipedia.org/wiki/Just-in-time_compilation
JSON / CSV	Text formats for measurements: JSON for structured data (one file per measurement), CSV as a table that e.g. Excel can open. https://en.wikipedia.org/wiki/JSON
Load balancing (dynamic)	Whoever finishes a tile fetches the next one. No core sits idle even though tiles take different amounts of time. https://en.wikipedia.org/wiki/Load_balancing_(computing)
Mandelbrot set	Famous fractal: for every pixel $z \to z^2 + c$ is repeated until the value "escapes" or the maximum count is reached. Every point is independent – ideal for parallelisation. https://en.wikipedia.org/wiki/Mandelbrot_set
Median	Middle value of a sorted series of measurements; less sensitive to outliers than the average. Used when several runs are made. https://en.wikipedia.org/wiki/Median
mpmath	Python library for arbitrary-precision arithmetic; serves as an independent cross-check of the Pi digits. https://mpmath.org/
NUMA	Non-Uniform Memory Access: on large systems memory is reached at different speeds depending on the core. The 3960X has one NUMA node. https://en.wikipedia.org/wiki/Non-uniform_memory_access
numba	Compiles selected Python functions to machine code on first call (JIT), making them about as fast as C. https://numba.pydata.org/
numpy	Python library for computing with whole arrays of numbers. Fast for simple array operations, but not ideal for Mandelbrot. https://numpy.org/
OpenBLAS	Math library inside numpy that starts its own threads. Limited to 1 thread per process in the monitor, otherwise every worker needs ~800 MB. https://www.openblas.net/

pip	Python's package manager; installs extensions such as numba or FastAPI. https://pip.pypa.io/
Process	A running program with its own memory. Processes run independently but can exchange data only indirectly. https://en.wikipedia.org/wiki/Process_(computing)
Processor group	Windows combines at most 64 threads into one group. Processors with more threads have several groups; a process normally runs in only one. https://learn.microsoft.com/en-us/windows/win32/procthread/processor-groups
ProcessPoolExecutor	Python building block that manages a pool of worker processes and distributes tasks to them. https://docs.python.org/3/library/concurrent.futures.html
psutil	Python library that queries load, memory and processes; supplies the values for the core display. https://psutil.readthedocs.io/
Python	Widely used scripting language; used here for the web server, control logic and two computation methods. https://www.python.org/
Quarantine	Isolated area of the antivirus software for suspicious files; they can be restored from there. https://en.wikipedia.org/wiki/Antivirus_software
Rust	Modern systems language with C-like speed; its compiler prevents typical mistakes when working with threads. https://www.rust-lang.org/
rustup / cargo	rustup installs and updates Rust; cargo builds Rust programs and downloads the libraries they need. https://rustup.rs/
Scaling, speedup, efficiency	Speedup = time with 1 worker ÷ time with n workers. Efficiency = speedup ÷ n (100 % = perfect). https://en.wikipedia.org/wiki/Speedup
Scheduler	Part of Windows that decides which thread runs on which core and when (placement "Windows decides"). https://en.wikipedia.org/wiki/Scheduling_(computing)
Seahorse valley	Well-known region of the Mandelbrot set between the main cardioid and the period-2 bulb; very finely branched and therefore compute-intensive. https://en.wikipedia.org/wiki/Mandelbrot_set
Self-contained single file	.NET program that carries the runtime inside a single .exe; runs without an installed .NET. https://learn.microsoft.com/en-us/dotnet/core/deploying/single-file/overview
SIMD	Single Instruction, Multiple Data: one instruction processes several numbers at once. https://en.wikipedia.org/wiki/Single_instruction,_multiple_data
SmartScreen	Windows protection that warns about unknown, unsigned programs ("Windows protected your PC"). https://learn.microsoft.com/en-us/windows/security/operating-system-security/virus-and-threat-protection/microsoft-defender-smartscreen/

SMT (simultaneous multithreading, Intel: "Hyper-Threading")	One core runs two instruction streams at once and fills gaps in its execution units. Usually gains 0–30 %, not 100 %. https://en.wikipedia.org/wiki/Simultaneous_multithreading
Thread (hardware thread, logical processor)	What Windows counts as a "CPU". With SMT every core offers 2 threads, i.e. 48 on the 3960X. https://en.wikipedia.org/wiki/Thread_(computing)
Thread (software)	Execution path inside a process; all threads share the memory and start much faster than processes. https://en.wikipedia.org/wiki/Thread_(computing)
Tile	Rectangular part of the image (default 128 × 128 pixels), the smallest unit of work for a worker.
vcomp140.dll / OpenMP	OpenMP is a standard that lets C/C++ programs distribute loops across threads; vcomp140.dll is Microsoft's runtime library for it and is included in the package. https://learn.microsoft.com/en-us/cpp/parallel/openmp/openmp-in-visual-cpp
Vector256	.NET data type for a 256-bit register (4 doubles); this is how C# uses AVX2. https://learn.microsoft.com/en-us/dotnet/api/system.runtime.intrinsics.vector256
VirusTotal	Free service that checks a file with about 70 antivirus engines at once. https://www.virustotal.com/
Visual Studio Build Tools	Microsoft's compilers for C, C++ and assembler without the full development environment; free of charge. https://visualstudio.microsoft.com/downloads/
WebSocket	Permanent connection between browser and server; the load values arrive through it every second without reloading. https://en.wikipedia.org/wiki/WebSocket
winget	Windows package manager; installs programs such as Python, .NET or Rust from the command line. https://learn.microsoft.com/en-us/windows/package-manager/winget/
Worker	A process or thread that works through tiles. The number of workers is set in the monitor (1–48).
x64	64-bit instruction set of today's Intel and AMD processors (also called x86-64 or AMD64). https://en.wikipedia.org/wiki/X86-64